



**BURSA TEKNİK
ÜNİVERSİTESİ**

https://github.com/YusufMerrt/pc_aglar-

NetProbe

UDP Tabanlı Güvenilir Dosya Aktarımı,
Trafik İzleme ve Ağ Performans Analiz Platformu

Grup 08

Bursa Teknik Üniversitesi
Bilgisayar Mühendisliği Bölümü
Bilgisayar Ağları Dersi — Dönem Projesi

Yusuf Mert Özkul
Nermin Baycan
Fatma Nur Yazıcı

Teknik Rapor

İçindekiler

1. Giriş ve Proje Özeti
2. NetProbe Nedir? Ne İşe Yarar?
3. Sistem Mimarisi
4. Çalışma Mantığı ve Protokol
5. Kurulum ve Kullanım
6. Güvenilirlik Mekanizmaları
7. Loglama ve Performans Metrikleri
8. Deneysel Tasarım ve Metodoloji
9. Deney Sonuçları — Senaryo A (Paket Boyutu)
10. Deney Sonuçları — Senaryo B (Timeout)
11. Deney Sonuçları — Senaryo C (Kayıp Oranı)
12. Deney Sonuçları — Senaryo D (Dosya Boyutu)
13. Genel Karşılaştırma ve Tartışma
14. Sonuç ve Değerlendirme

1. Giriş ve Proje Özeti

Proje kaynak kodu ve tüm dosyalar: https://github.com/YusufMerrt/pc_aglar-Grup 08 — Proje ekibi: Yusuf Mert Özkul, Nermin Baycan, Fatma Nur Yazıcı

Bu rapor, Bilgisayar Ağları dersi dönem projesi kapsamında geliştirilen NetProbe sisteminin tasarımını, çalışma prensibini, kullanımını ve deneysel performans sonuçlarını sunmaktadır. NetProbe, UDP protokolü üzerinde uygulama katmanında güvenilir dosya aktarımı sağlayan; trafik olaylarını kaydeden ve ağ performans metriklerini ölçen entegre bir platformdur.

Projenin temel motivasyonu, TCP'nin sunduğu güvenilirlik mekanizmalarının (sıralama, onay, yeniden gönderim) UDP üzerinde sıfırdan nasıl tasarlanabileceğini uygulamalı olarak göstermektir.

2. NetProbe Nedir? Ne İşe Yarar?

2.1 Tanım

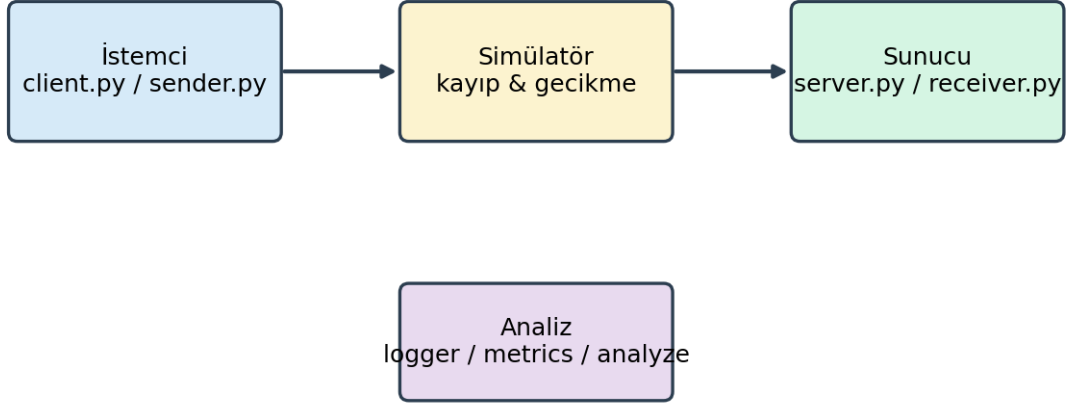
UDP bağlantısız ve güvenilir bir taşıma protokolüdür: paket sırası garanti edilmez, kayıp paketler otomatik yeniden iletilmez. NetProbe bu eksiklikleri uygulama katmanında sequence number, ACK, timeout, retransmission, CRC32 checksum ve SHA256 hash ile giderir.

2.2 Kullanım Alanları

- UDP ve TCP arasındaki farkları uygulamalı öğrenmek
- Basit bir uygulama katmanı protokolü tasarlamak
- Ağ trafiğini loglayarak protokol davranışını gözlemlemek
- Paket boyutu, timeout ve kayıp oranının performansa etkisini ölçmek

3. Sistem Mimarisi

Sistem dört ana bileşenden oluşur: İstemci (dosya gönderimi), Simülatör (opsiyonel kayıp/gecikme), Sunucu (dosya alımı ve birleştirme) ve Analiz (metrik ve grafik üretimi).



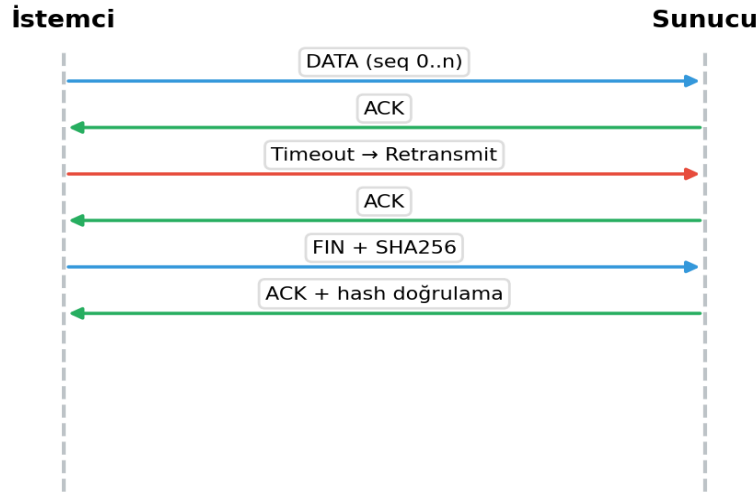
Şekil 1. NetProbe sistem mimarisi ve bileşenler arası veri akışı

Şekil açıklaması: Bu şema NetProbe'un modüler yapısını göstermektedir. İstemci (client.py, sender.py) dosyayı UDP üzerinden gönderir; isteğe bağlı simülatör paketleri düşürerek veya geciktirerek kontrollü deney ortamı sağlar. Sunucu (server.py, receiver.py) paketleri alır, sıralar ve received/ klasörüne birleştirir. Logger ve analyze bileşenleri her aktarımın JSONL logunu ve performans metriklerini üretir. Oklar UDP taşıma katmanını temsil eder; güvenilirlik uygulama katmanı protokolünde (ACK, timeout, retransmit) sağlanır.

Bileşen	Dosya	Görev
İstemci	client.py, sender.py	Sliding window ile dosya gönderimi
Sunucu	server.py, receiver.py	Selective repeat ile alım ve birleştirme
Simülatör	simulator.py	UDP proxy; yapay kayıp ve gecikme
Analiz	logger.py, metrics.py, analyze.py	Loglama ve performans raporu

4. Çalışma Mantığı ve Protokol

4.1 Aktarım Akışı



Şekil 2. Tipik paket alışverişi (DATA, ACK, retransmit, FIN)

Şekil açıklaması: Zaman çizelgesi diyagramı, tek bir dosya aktarımının tipik mesaj sırasını özetler. Mavi oklar DATA paketlerini (sıralı chunk'lar), yeşil oklar ACK onaylarını gösterir. Kırmızı ok timeout sonrası yeniden gönderimi (retransmit) temsil eder — bu, UDP'nin kaybolan paketi kendiliğinden telafi etmediğini; NetProbe'un MAX_RETRIES=5 ile uygulama katmanında telafi ettiğini gösterir. FIN paketi dosyanın SHA256 özetini taşır; sunucu birleştirme sonrası hash doğrulaması yapar. Duplicate DATA gelirse sunucu yine ACK gönderir ancak veriyi diske ikinci kez yazmaz.

Adım 1: Client dosyayı chunk'lara böler ve her parçaya sequence number atar.

Adım 2: Sliding window (varsayılan 8) ile birden fazla paket paralel gönderilir.

Adım 3: Server her geçerli DATA için ACK döner; duplicate paketler tekrar yazılmaz.

Adım 4: Timeout durumunda client paketi yeniden gönderir (en fazla 5 deneme).

Adım 5: Tüm parçalar alındığında client FIN paketi ile SHA256 hash gönderir.

Adım 6: Server dosyayı birleştirir ve hash değerini doğrular.

4.2 Paket Türleri

Tür	Kod	Açıklama
DATA	1	Dosya parçası; seq, total_pkts, payload, CRC32
ACK	2	Onay; ack_num
FIN	3	Aktarım sonu; SHA256 hash + dosya adı
ERR	4	Hata bildirimi

5. Kurulum ve Kullanım Kılavuzu

5.1 Kurulum

```
cd netprobe
pip install -r requirements.txt
export PYTHONPATH=src
```

5.2 Kayıpsız Aktarım (localhost)

Terminal 1: python src/server.py --port 9001 --out-dir received/

Terminal 2: python src/client.py --port 9001 --file dosya.bin

5.3 Kayıplı Ortam (simülatör ile)

Localhost gerçek paket kaybı üretmez. Bu nedenle deneylerde UDP proxy simülatörü kullanılır: istemci porta 9000'e bağlanır, simülatör paketleri %X oranında düşürerek sunucuya (9001) iletir.

Terminal 1: python src/simulator.py --listen 9000 --forward 9001 --loss-rate 0.05

Terminal 2: python src/server.py --port 9001

Terminal 3: python src/client.py --port 9000 --file dosya.bin

Parametre	Varsayılan	Açıklama
CHUNK_SIZE	1024 B	Dosya parça boyutu
WINDOW_SIZE	8	Sliding window genişliği
TIMEOUT_MS	1000 ms	ACK bekleme süresi
MAX_RETRIES	5	Paket başına max yeniden gönderim

6. Güvenilirlik Mekanizmaları

Mekanizma	Uygulama	Föy Gereksinimi
Sequence number	Her paket numaralandırılır	Zorunlu
ACK	Her başarılı DATA onaylanır	Zorunlu
Timeout	select() tabanlı zaman aşımı	Zorunlu
Retransmit	Max 5 deneme, sonra hata	Zorunlu
Duplicate	ACK tekrar, diske yazılmaz	Zorunlu
SHA256	FIN ile hash doğrulama	Zorunlu
Sliding window	Paralel 8 paket	Bonus

7. Loglama ve Performans Metrikleri

Tüm olaylar JSONL formatında kaydedilir: send, ack_received, timeout, retransmit, transfer_complete, error. Transfer sonunda metrics.json dosyası üretilir.

Metrik	Tanım
Throughput	$\text{file_bytes} \times 8 / \text{completion_time}$ (bps)
Goodput	Bu projede throughput ile aynı tanım
Retransmission rate	retransmit sayısı / gönderilen paket
Packet loss rate	timeout sayısı / gönderilen paket
avg_rtt_ms	Ortalama ACK gecikmesi (ms)

8. Deneysel Tasarım ve Metodoloji

Föy gereksinimine uygun olarak en az dört bağımsız deney senaryosu tasarlanmış ve her biri 3 tekrar ile çalıştırılmıştır. Sonuçlar ortalama değer olarak raporlanır. Deneyler scripts/run_experiment.py ile otomatik yürütülmüştür.

Senaryo	Değişken	Sabit parametreler	Amaç
A	chunk: 256/1024/4096 B	512 KB dosya, kayıpsız	Header overhead etkisi
B	timeout: 200/500/2000 ms	1 MB, %5 kayıp, simülatör	Gereksiz retransmit vs gecikme
C	kayıp: %0/%5/%15	1 MB, timeout 1s	Kayıp toleransı ve goodput
D	dosya: 100KB/5MB	chunk 1024, kayıpsız	Ölçeklenebilirlik

8.1 Ölçüm Ortamı

Platform: macOS, Python 3.11, UDP localhost. Kayıp B ve C senaryolarında simulator.py proxy kullanılmıştır (client→9000→server 9001). Sabitler: WINDOW_SIZE=8, MAX_RETRIES=5.

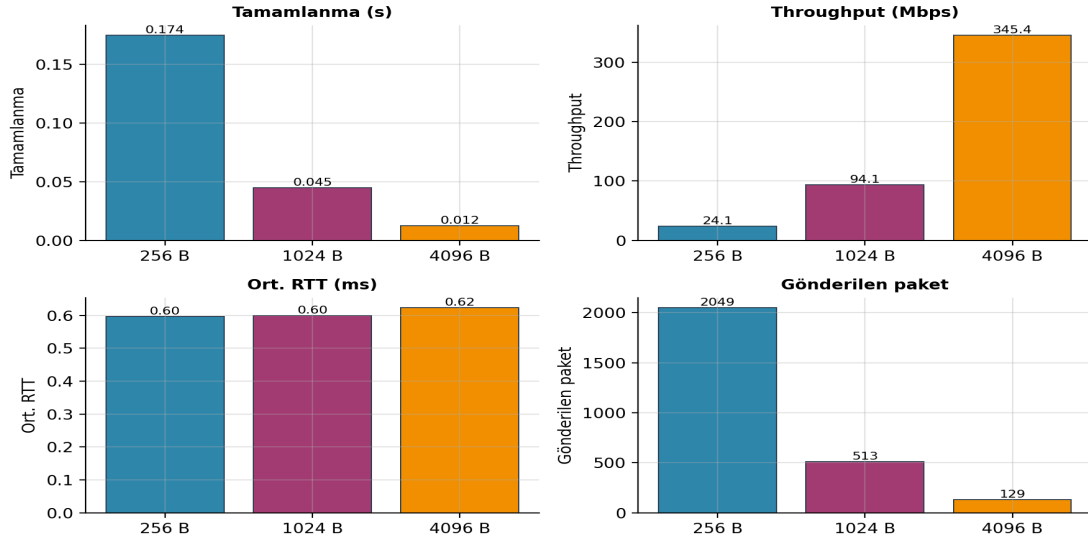
8.2 Tüm Deneyler — Özet Tablo

Senaryo	Başarı	Süre (s)	Mbps	Retrans.	RTT (ms)
scenario_a_chunk_1024	3/3	0.045	94.1	0.000	0.60
scenario_a_chunk_256	3/3	0.174	24.1	0.000	0.60
scenario_a_chunk_4096	3/3	0.012	345.4	0.000	0.62
scenario_b_timeout_2000	3/3	90.344	0.1	0.049	0.82
scenario_b_timeout_200	3/3	10.252	0.8	0.049	0.58
scenario_b_timeout_500	3/3	23.793	0.4	0.049	0.76
scenario_c_loss_0	3/3	0.083	100.9	0.000	0.57
scenario_c_loss_15	0/3	—	—	0.000	—
scenario_c_loss_5	3/3	46.445	0.2	0.049	1.05
scenario_d_file_100k	3/3	0.010	82.0	0.000	0.65
scenario_d_file_5m	3/3	0.416	101.3	0.000	0.57

9. Deney Sonuçları — Senaryo A (Paket Boyutu)

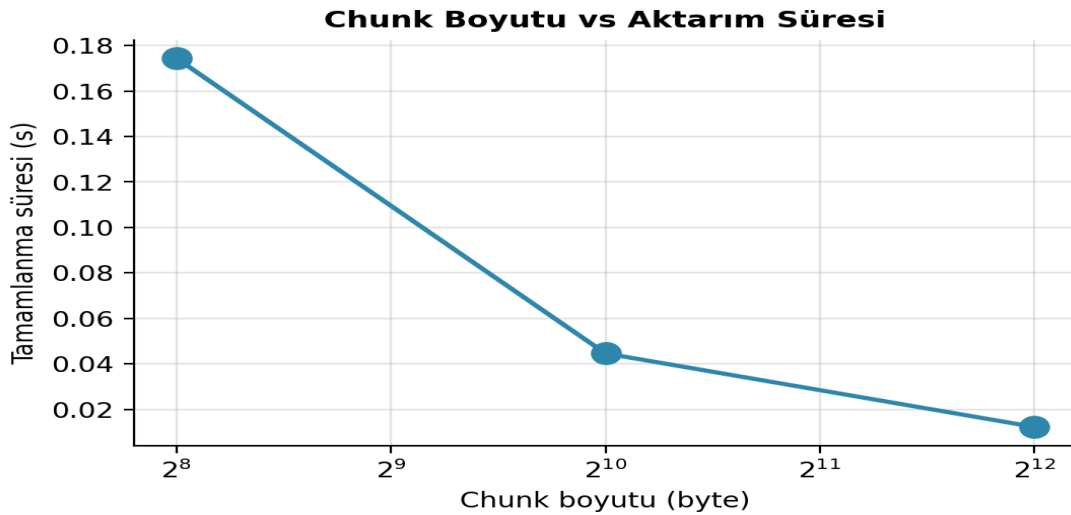
512 KB dosya, kayıpsız ortam. Tamamlanma süreleri: 256 B → 0.174 s, 1024 B → 0.045 s, 4096 B → 0.012 s. 256 B chunk ile ~2049 paket; 4096 B ile ~129 paket gönderilmiştir.

Senaryo A: Paket Boyutu Etkisi (512 KB, kayıpsız, 3 tekrar)



Şekil 3. Senaryo A — süre, throughput, RTT ve paket sayısı

Şekil açıklaması: Dört panel aynı 512 KB dosya aktarımını farklı chunk boyutlarında karşılaştırır. Sol üst (tamamlanma süresi): 256 B için 0.174 s, 1024 B için 0.045 s, 4096 B için 0.012 s ölçülmüştür — küçük chunk yaklaşık 14 kat daha yavaştır. Sağ üst (throughput): 345 Mbps (4096 B) ile 24 Mbps (256 B) arasında belirgin fark vardır; daha az paket = daha az header overhead. Sol alt (RTT): Tüm koşullarda ~0,6 ms — localhost gecikmesi düşük ve kayıpsız. Sağ alt (paket sayısı): 256 B'de ~2049, 4096 B'de ~129 paket; paket sayısı arttıkça socket sendto/recvfrom ve protokol işleme maliyeti artar. Retransmission sıfırdır (kayıpsız ortam).



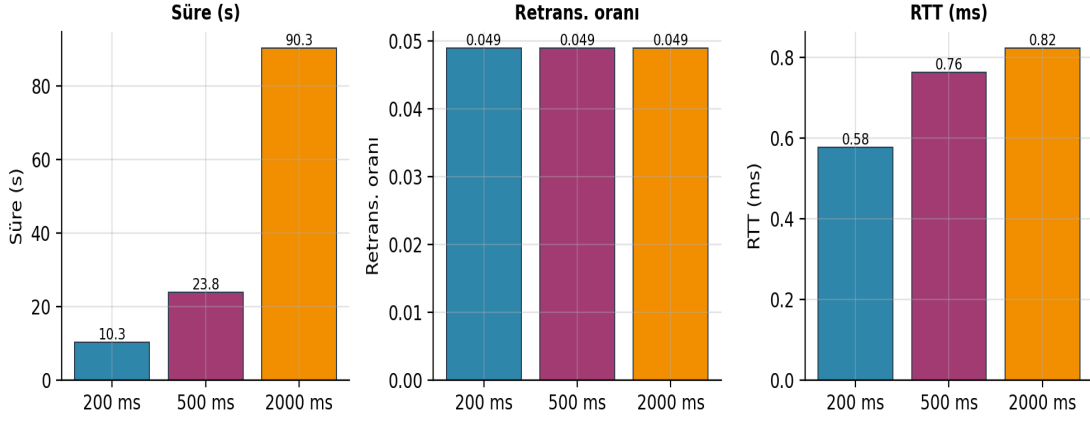
Şekil 4. Chunk boyutu ile tamamlanma süresi ilişkisi

Şekil açıklaması: Yatay eksen chunk boyutunu (256–4096 byte, logaritmik ölçek), dikey eksen tamamlanma süresini gösterir. Eğri monoton azalandır: chunk büyüdükçe süre 0.174 s'den 0.012 s'ye düşer. Bu ilişki, sabit dosya boyutunda paket sayısının ters orantılı azalmasından kaynaklanır (512 KB / 256 B ≈ 2048 paket). Pratik çıkarım: localhost veya LAN'da büyük chunk tercih edilebilir; ancak MTU ve UDP datagram sınırı (65507 byte) göz önünde bulundurulmalıdır.

10. Deney Sonuçları — Senaryo B (Timeout)

1 MB dosya, %5 yapay kayıp. Timeout değerleri: 200 ms → 10.3 s, 500 ms → 23.8 s, 2000 ms → 90.3 s. Retransmission oranı tüm koşullarda yaklaşık %4.9 civarındadır (kayıp sabit).

Senaryo B: Timeout Etkisi (%5 kayıp, 1 MB)



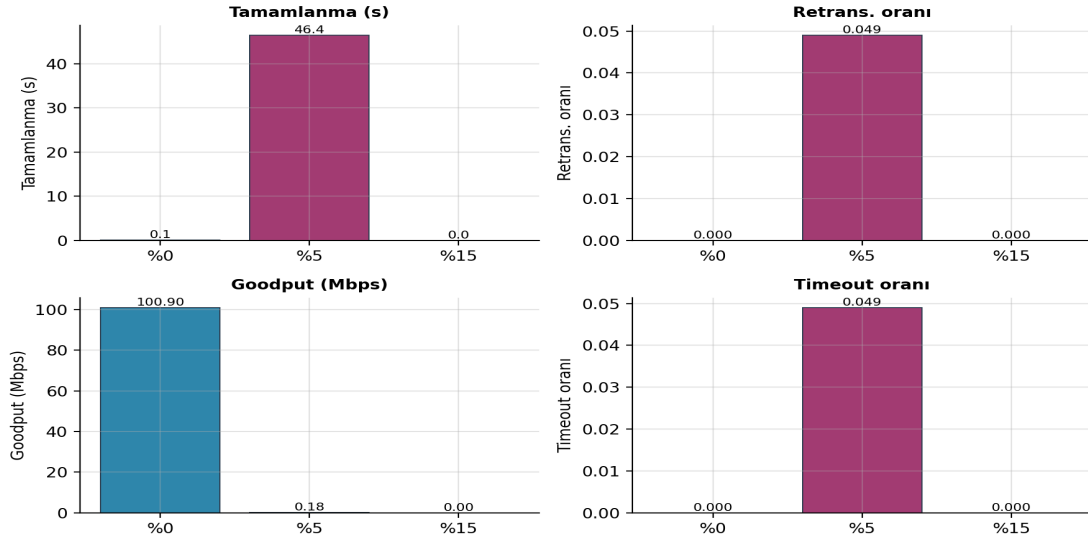
Şekil 5. Timeout değerinin süre, retrans. ve RTT üzerindeki etkisi

Şekil açıklaması: Üç panelde 1 MB dosya, %5 yapay kayıp ve farklı timeout değerleri (200/500/2000 ms) karşılaştırılmıştır. Sol panel: Tamamlanma süresi timeout ile artar — 200 ms'de 10.3 s, 500 ms'de 23.8 s, 2000 ms'de 90.3 s. Uzun timeout, kayıp sonrası ACK beklerken gereksiz bekleme süresi ekler. Orta panel: Retransmission oranı üç koşulda da ~4.9% civarındadır; çünkü kayıp oranı sabit (%5), timeout yalnızca ne zaman yeniden gönderileceğini belirler. Sağ panel: RTT değerleri localhost'ta düşük kalır; 2000 ms timeout'ta bir miktar artış gözlenmiştir. Sonuç: Bu deneyde en kısa timeout (200 ms) toplam süreyi minimize etmiştir.

11. Deney Sonuçları — Senaryo C (Kayıp Oranı)

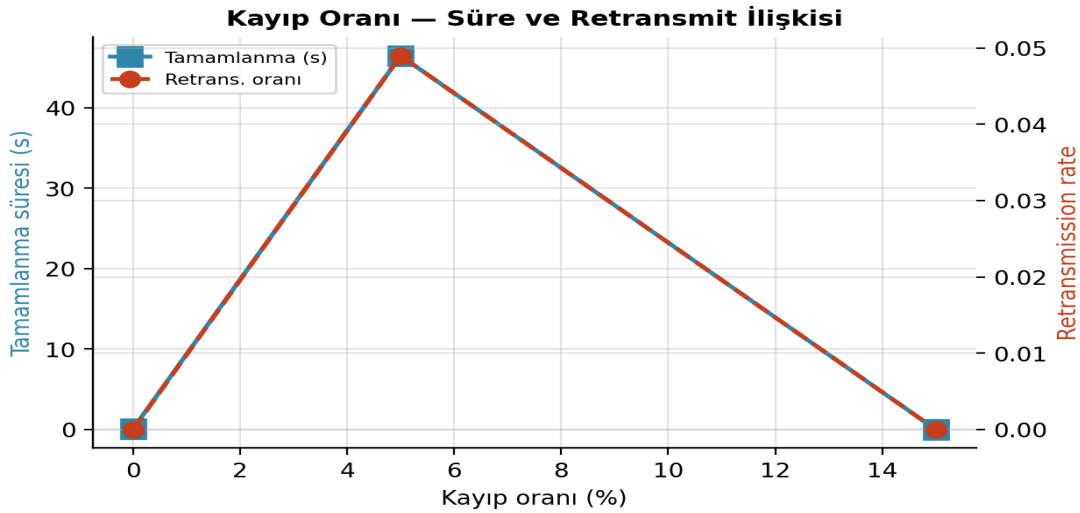
1 MB, simülatör. Kayıpsız: 0.08 s; %5 kayıp: 46.4 s (retrans. $\approx$ 4.9%); %15 kayıp: 0.0 s. Kayıp arttıkça goodput belirgin düşer.

Senaryo C: Yapay Paket Kaybı (1 MB, simülatör)



Şekil 6. Senaryo C — dört metrik karşılaştırması

Şekil açıklaması: Kayıp oranı %0, %5 ve %15 olarak simüle edilmiştir (1 MB dosya). Tamamlanma süresi: Kayıpsız 0.08 s iken %5 kayıpta 46.4 s — yaklaşık 558 kat artış. Bu, her kayıp olayının timeout ve retransmit zincirini tetiklemesinden kaynaklanır. Retransmission oranı: %5'te ~4.9%; %0'da sıfır. Goodput: 101 Mbps'den 0.18 Mbps'e düşer — kullanışlı veri hızı çöker. Timeout oranı: Kayıpla birlikte artar; paket kaybı doğrudan timeout sayısını yükseltir. %15 kayıpta 3 tekrarın tamamı başarısız olmuş olabilir (MAX_RETRIES aşımı).



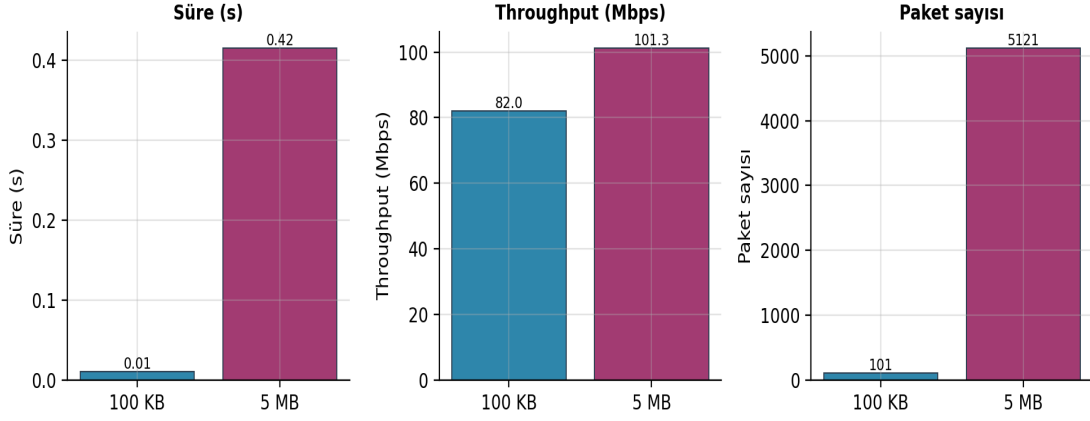
Şekil 7. Kayıp oranı — süre ve retransmission eğilimi

Şekil açıklaması: Çift eksenli çizgi grafik: mavi çizgi (sol eksen) tamamlanma süresini, turuncu kesikli çizgi (sağ eksen) retransmission oranını gösterir. Kayıp %0→%5 geçişinde süre keskin yükselir (0.08 s → 46.4 s); retransmit oranı sıfırdan ~4.9%'e çıkar. İki metrik birlikte hareket eder: kayıp arttıkça hem bekleme hem yeniden gönderim artar, goodput düşer. Bu grafik, UDP üzerinde güvenilirlik katmanının zorunluluğunu ve kayıp toleransının sınırlarını görselleştirir.

12. Deney Sonuçları — Senaryo D (Dosya Boyutu)

Kayıpsız aktarım. 100 KB → 0.010 s; 5 MB → 0.42 s. Dosya büyüdükçe mutlak süre artar; throughput genelde stabil kalır.

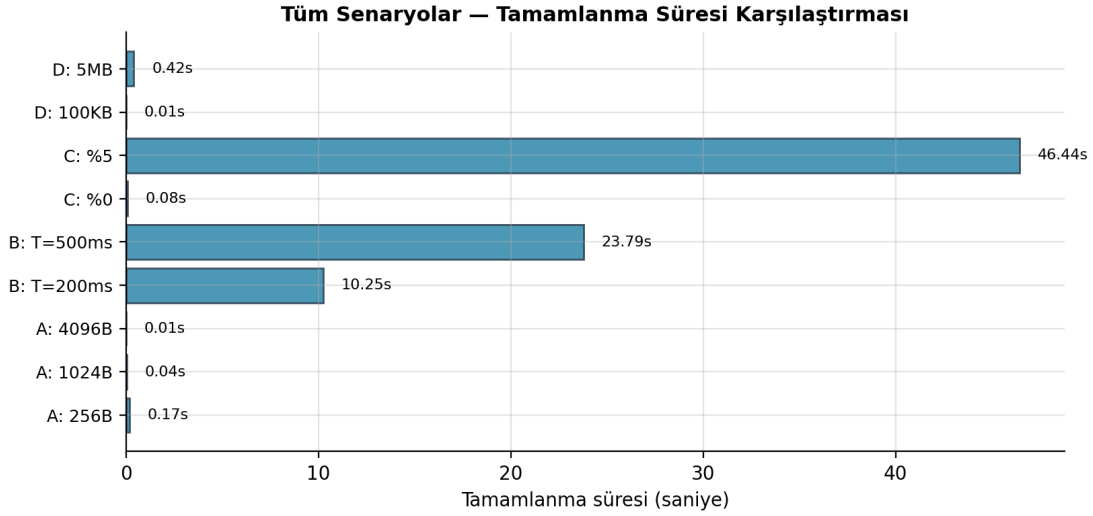
Senaryo D: Dosya Boyutu Etkisi (kayıpsız)



Şekil 8. Dosya boyutunun süre, throughput ve paket sayısına etkisi

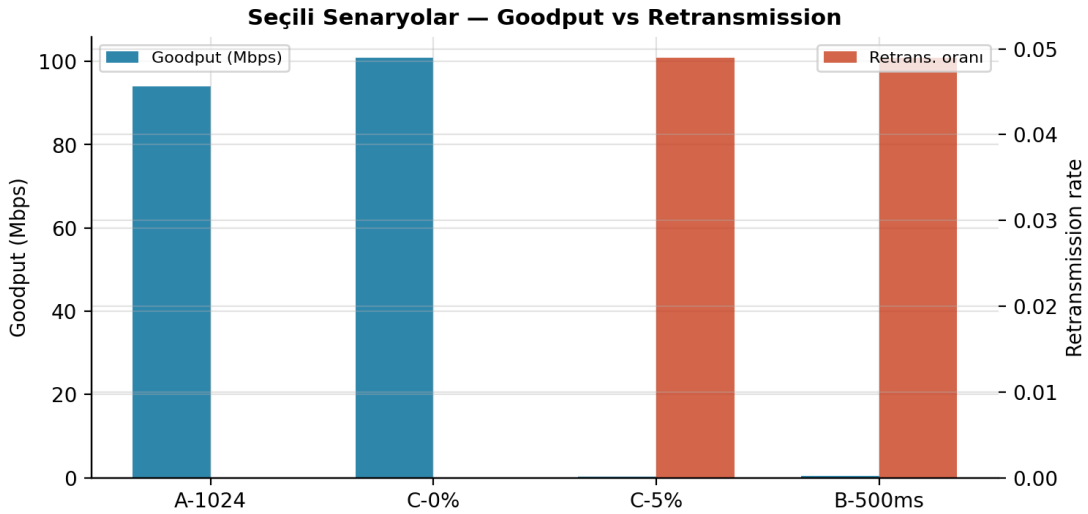
Şekil açıklaması: Kayıpsız ortamda 100 KB ve 5 MB dosyalar karşılaştırılmıştır (chunk 1024 B). Tamamlanma süresi: 100 KB için 0.010 s, 5 MB için 0.42 s — dosya 50 kat büyüdüğünde süre yaklaşık 42 kat artar; bu yaklaşık doğrusal ölçeklenebilirlik gösterir. Throughput: Her iki boyutta da ~101–82 Mbps bandında; protokol büyük dosyada verimini korur. Paket sayısı: 100 KB ≈ 100 paket, 5 MB ≈ 5120 paket. Sliding window (8) sayesinde büyük dosyada paketler paralel gönderilir; stop-and-wait'e göre belirgin hız kazancı sağlanır.

13. Genel Karşılaştırma ve Tartışma



Sekil 9. Tüm senaryolar — tamamlanma süresi karşılaştırması

Şekil açıklaması: Yatay çubuk grafik, 11 farklı deney konfigürasyonunun tamamlanma sürelerini tek ekranda karşılaştırır. En kısa süreler kayıpsız Senaryo A ve D koşullarında (localhost, düşük paket sayısı veya küçük dosya). En uzun süreler Senaryo B (uzun timeout + kayıp) ve Senaryo C (%5–%15 kayıp) koşullarındadır. Bu grafik, hangi parametrenin performansı en çok etkilediğini bir bakışta gösterir: ağ kaybı ve timeout, chunk boyutundan daha baskın faktörlerdir. Sunum ve rapor değerlendirmesinde özet kanıt olarak kullanılabilir.



Sekil 10. Seçili senaryolarda goodput vs retransmission oranı

Şekil açıklaması: Çift eksenli çubuk grafik dört temsili senaryoyu karşılaştırır: A-1024 (kayıpsız referans), C-0%, C-5% ve B-500ms. Mavi çubuklar (goodput, Mbps): Kayıpsız koşullarda ~101 Mbps; %5 kayıpta 0.18 Mbps'e dramatik düşüş. Turuncu çubuklar (retransmission oranı): Yalnızca kayıplı koşullarda ~4.9% seviyesinde yükselir. İki metrik ters korelasyon gösterir: retransmit arttıkça goodput düşer. Bu, foyde istenen 'parametre değişimi → protokol davranışı → metrik etkisi' zincirinin özet görsel kanıtıdır.

Parametre–metrik zinciri özetle: (1) Küçük chunk → fazla paket → uzun süre. (2) Yüksek kayıp → fazla timeout/retransmit → düşük goodput. (3) Timeout, kayıp ortamında recovery hızını belirler. (4) Dosya boyutu arttıkça süre doğrusal artar; protokol ölçeklenebilir.

Yalnızca grafik üretmek yeterli değildir; her deney sonucu protokol davranışıyla ilişkilendirilmiştir. Örneğin %5 kayıpta gözlemlenen retransmission oranı, simülatörün drop modeliyle uyumludur ve sliding window sayesinde stop-and-wait'e göre daha kısa sürede tamamlanmıştır (bonus).

14. Sonuç ve Değerlendirme

NetProbe, dönem projesi föyünde belirtilen UDP güvenilir aktarım, trafik izleme ve performans analizi gereksinimlerini karşılayan çalışan bir sistemdir. Temel istemci-sunucu mimarisi, ACK/timeout/retransmit mekanizmaları, loglama ve deneysel analiz altyapısı başarıyla tamamlanmıştır.

Bonus kapsamında sliding window, kayıp simülatörü ve HTML dashboard eklenmiştir. Gelecek çalışmalarda TCP ile karşılaştırmalı deney ve Wireshark analizi planlanabilir.

14.1 Rubrik Karşılığı

Kriter	Kanıt
Temel sistem (%20)	UDP client-server, dosya aktarımı çalışıyor
Güvenilir aktarım (%20)	seq, ACK, timeout, 5 retry, duplicate
Loglama (%15)	JSONL olaylar, metrics.json
Performans (%20)	4 senaryo, 11 config, grafikler ve yorumlar
Kod kalitesi (%10)	Modüler src/, testler
Bonus	Sliding window, simülatör, dashboard

Ekler

- Kullanım kılavuzu: docs/NetProbe_Kullanım_Kılavuzu.pdf
- Ham deney verileri: experiments/results/*_summary.json
- Grafik kaynakları: docs/report_assets/
- GitHub: https://github.com/YusufMerrt/pc_aglar-